

Gruggle JavaScript API Guide

December 13, 2025

1 Overview

Gruggle is a single-file DOT loader/transformer that can be used both as a CLI and as a programmatic API from Node.js. The CLI remains unchanged; when `gruggle.js` is imported via `require`, no CLI code runs and the API is exposed.

2 Getting Started

- Ensure Node.js is available.
- Place `gruggle.js` in your project (or install it alongside your code).
- Use either the CLI (`node gruggle.js ...`) or import it:

```
const {
  Gruggle,
  parse,
  serialize,
  applySubcommands,
  applyOne,
  parseChainString,
  emptyGraph,
  mergeGraphs,
} = require('./gruggle');
```

3 CLI vs Library

- CLI: invoked via `node gruggle.js ...`; supports all documented subcommands.
- Library: when required, the module exports API objects; `main()` is not executed (checked via `require.main === module`).

4 Core Concepts

- **Graph object:** plain JS object containing nodes (Map), edges (array), defaults, and attributes.
- **Subcommands:** the same operations used by the CLI; they can be invoked programmatically via `applyOne` or `applySubcommands`.
- **Chain strings:** CLI-style quoted command strings parsed by `parseChainString`.

5 Class: Gruggle

Convenience wrapper around the core helpers.

Construction

`new Gruggle(graph = emptyGraph())`

Start from an existing graph object or a fresh empty graph.

`Gruggle.fromDot(text, edgeMode = 'curved')`

Parse DOT text; `edgeMode` can be `'curved'` (default) or `'ortho'/'rectilinear'` (sets `splines=ortho`).

`Gruggle.fromFiles(files, edgeMode = 'curved')`

Parse and merge a list of DOT files, then set edge mode.

Methods

`chain(commands)`

Run one or more subcommands using the `chain` wrapper. Accepts a single chain string (e.g., `"add-nodes n:1:3"`) or an array of chain strings or token arrays. Returns `this`.

`apply(tokens)`

Apply a token list directly (e.g., `['add-nodes', 'n:1:3']`); returns `this`.

`toDot()`

Serialize the current graph to DOT text (obeys keep/remove flags).

`toSvg(outFile)`

Emit SVG via Graphviz `dot`, writing to `outFile`; returns `this`.

`value()`

Return the underlying graph object (for advanced inspection/manipulation).

6 Helper Functions

`parse(text)`

Parse DOT text into a graph object.

`serialize(graph)`

Serialize a graph to DOT text, honoring keep/remove semantics.

`applyOne(graph, tokens)`

Apply a single subcommand token array to a graph.

`applySubcommands(graph, tokens)`

Apply a token array (often starting with a subcommand, e.g., `['add-nodes', 'n:1:3']` or `['chain', 'cmd1', 'cmd2']`).

`parseChainString(str)`

Parse a CLI-style chain string into tokens, handling quoted segments.

`emptyGraph()`

Create a new empty digraph with defaults.

`mergeGraphs(graphs)`

Merge multiple graphs (as the CLI does for multiple `-i` inputs).

7 Usage Examples

Build and serialize

```
const { Gruggle } = require('./gruggle');
const gm = new Gruggle();
gm.apply(['add-nodes', 'n:1:3'])
  .apply(['add-edges', 'lbl', 'n.*', 'n.*']);
console.log(gm.toDot());
```

Chain with CLI-style strings

```
const { Gruggle, parseChainString } = require('./gruggle');
const gm = Gruggle.fromFiles(['graph1.g']);
gm.chain(parseChainString('add-nodes▯extra:1:2'))
  .chain(['add-edges', 'note', 'extra.*', 'n.*'])
  .toSvg('out.svg');
```

Direct helpers without Gruggle

```
const { parse, applySubcommands, serialize, parseChainString } = require('./gruggle');
let g = parse('digraph▯G▯{▯a▯->▯b;▯}');
g = applySubcommands(g, ['add-nodes', 'c']);
g = applySubcommands(g, ['add-edges', 'lbl', 'a.*', 'c']);
console.log(serialize(g));
```

8 Notes and Compatibility

- The CLI behavior is unchanged; all existing tests and subcommands remain valid.
- The API lives in the same single file; distribution remains a single `gruggle.js`.
- `toSvg` requires Graphviz `dot` on `PATH`, same as the CLI.
- Chain parsing rules match the CLI; use `parseChainString` when you want to feed a quoted command string.